DeployGuard

Deployment Issue Resolution and Current Readiness Report

Team-facing summary of the failures encountered, root causes, permanent fixes, verification completed, and the exact stage the product has reached.

Current position
The major repository-analysis and deployment-state problems are resolved and regression-tested locally. The latest persistence conflict has been removed locally. The next milestone is to publish and pin the corrected immutable reusable workflow, then perform one bounded live deployment through Terraform apply -> ECS -> ALB -> LIVE.

ANALYZE	BUILD	PUBLISH	PROVISION	RUN	PROVE LIVE
repo + BuildPlan	Docker + scan	ECR digest	Terraform	ECS + ALB	health + evidence

Prepared for: DeployGuard project team

Report date: 10 August 2026

1. Executive Summary

The recent failures were not evidence that the overall Terraform -> ECS -> ALB deployment architecture was fundamentally wrong. Most failures came from inconsistent state interpretation across layers: environment identity, repository/build information, and especially persistence/recovery state were being interpreted independently in different parts of the system.

- Environment identity was inconsistent across deployment, variables, snapshots and locks. This was unified under one canonical environment identifier.
- Deployment stages independently inferred application/runtime details. This was replaced by one authoritative persisted BuildPlan.
- Framework support and Docker/runtime templates were inconsistent. Deterministic framework detectors and certified templates were added.
- The reusable workflow received only derived BuildPlan fields. It now receives and validates the complete immutable BuildPlan before Docker or Terraform work.
- The latest live failure was caused by stale database secrets being treated as proof that persistent database data previously existed. That legacy inference has now been removed locally.
- No AWS mutation has been performed after the latest fix. Live certification is the remaining milestone, not another architecture redesign.

2. Issues Encountered and How They Were Resolved

Issue	Type	Root cause / what happened	Resolution	Status
Environment identity mismatch	Architecture/integration	Deployment identity used dev while other state used production/development.	Introduced one immutable canonical environmentName across persistence, readiness, snapshots, locks, workflow, Terraform/ECS and release evidence.	RESOLVED
Independent deployment interpretation	Design/integration	Different stages interpreted deployment profile/readiness independently.	Introduced a persisted authoritative BuildPlan and exact readiness states; Docker, preflight, workflow and requirements consume the same plan.	RESOLVED
Framework/template inconsistency	Implementation	Framework detection was monolithic and several mainstream frameworks/modes lacked aligned strategies.	Added deterministic detector interfaces, 24 certified framework/mode fixtures, secure runtime templates and bounded Python AST inspection.	RESOLVED
Incomplete workflow contract	Integration	Workflow received derived BuildPlan fields rather than the complete authoritative plan.	Dispatches full immutable BuildPlan and validates repo, branch, commit, app root, port, health path, template and output before build/Terraform.	RESOLVED
Control-plane runtime mismatch	Implementation/operations	DeployGuard control plane was inconsistently using Node 20 while the supported runtime moved to Node 22.	Aligned product runtime/build/start/verify to Node 22 without forcing customer applications to Node 22.	RESOLVED
Legacy DB persistence inference	Implementation/integration	Any managed DB secret could set DATABASE_SECRET_PRESENT=true; missing EFS was then interpreted as lost database data.	Workflow now uses immutable deploymentContext.persistentState exclusively; secrets are identities, not proof that a persistent generation existed.	RESOLVED LOCAL
Fresh/retry deployment blocked	Consequence	Fresh and first-retry deployments could be blocked by stale DB/EFS evidence.	For authoritative NONE, retained-EFS adoption/recovery checks are skipped; real PERSISTENT generations remain protected.	REGRESSION PASS
Live AWS end-to-end proof	Verification gap	Local and contract tests cannot prove the actual Terraform -> ECS -> ALB -> public endpoint path.	Publish/pin the fixed workflow, then run one bounded normal deployment and capture live evidence.	NEXT

3. Latest Deployment Failure - Simple Explanation

> **Observed failure**
> Terraform initialized successfully, then the workflow stopped with: "Managed database data is unavailable and no retained filesystem was found." This was not a Terraform initialization error. It was a recovery/persistence decision made after initialization.

What the old workflow incorrectly did

OLD SECRET	EFS	ASSUMPTION	RESULT
exists	missing	old DB data existed	block deploy

- A database secret could remain from a failed or partial attempt.
- The workflow treated that secret as proof that a real persistent database generation had existed.
- If EFS was missing, it assumed database data had been lost and blocked both fresh deployment and retry.

Why that inference was wrong

A secret is configuration identity, not evidence of successful persistent data. A password/URL secret can exist even when the database and filesystem were never successfully created. The workflow was effectively seeing an old key and assuming a completed house had existed.

What the corrected workflow does now

BACKEND CONTEXT	NONE	PERSISTENT	POLICY
persistentState	fresh/first retry	real prior data	allow or protect

- persistentState = NONE: do not perform retained-EFS recovery inference; create/reconcile DB/EFS normally if the BuildPlan requires them.
- persistentState = PERSISTENT: verify retained storage/recovery evidence and block genuinely destructive or unrecoverable changes.
- Secrets and bindings are resource identities/evidence inputs, but they cannot independently prove that persistent production data existed.
- Internally inconsistent immutable deployment contexts are rejected before AWS access.

4. Permanent Reliability Rules Now in Place

One canonical environment: All project state, variables, snapshots, locks, workflow inputs, Terraform/ECS values and release evidence use the same environment identity.

One authoritative BuildPlan: Repository facts are detected once, persisted, fingerprinted and consumed by readiness, Docker, workflow and runtime configuration.

One authoritative deployment context: FRESH / UPDATE / RETRY / RESTORE / RESET_FRESH and persistentState are resolved before dispatch and carried immutably into the workflow.

Stable foundation, immutable release: Foundation resources are reconciled/reused; each deployment uses a unique image digest and release/task-definition identity.

Old attempts do not become current truth: Historical attempts remain evidence. Retry creates a new attempt. Old secret/binding residue cannot automatically control recovery state.

Fail closed only on real risk: Unsupported stacks, stale commits, unsafe Dockerfiles, ownership conflicts and genuinely missing protected data still block.

Health evidence defines success: A deployment should only become LIVE after the runtime/ALB/public health evidence and release result are validated.

5. Verification Completed

Verification	Result
Canonical environment resolution	PASS
Configuration ownership / reserved boundary	PASS
Authoritative BuildPlan	PASS
Deployability / preflight	PASS
24 mainstream framework/mode fixtures	PASS
Docker templates and non-root runtime smoke	PASS
GitHub Actions runtime/configuration contract	PASS
Repository-analysis certification	PASS
Terraform export / pre-plan reconciliation	PASS
Database-tier integration	PASS
Fresh DB deployment regression	PASS
Retry after failed first deployment	PASS
Stale secret / binding regression	PASS
Existing persistent deployment protection	PASS
Missing persistent data without backup	PASS - correctly blocked
Missing persistent data with backup	PASS - restore required
Repeated deployments	PASS
Reset-fresh semantics	PASS
Destroy retention	PASS
Backend production build under Node 22	PASS
Frontend production build under Node 22	PASS
Local product verification / readiness	PASS
Live hosted GitHub Actions + AWS deployment after latest fix	NOT YET RUN

6. Current Stage of the Product

Current stage: Pre-live deployment certification

Repository analysis, BuildPlan, framework certification, workflow contracts, local startup and the persistence/recovery regression fix are implemented and locally verified. The latest recovery fix exists locally but must be published as a new immutable reusable-workflow commit and pinned before the next customer/test deployment.

Stage	Status	Meaning
Repository analysis and BuildPlan	COMPLETE	Detection, readiness, Docker/runtime contract and framework fixtures are certified.
Workflow contract and safety	COMPLETE	Immutable BuildPlan and deployment context are validated before build/Terraform.
Persistence/recovery conflict fix	COMPLETE LOCALLY	Legacy secret-based inference removed; regression suite passes.
Publish + pin corrected workflow	NEXT	Create new immutable workflow commit SHA and update DeployGuard pin.
One normal live deployment	NEXT	Prove Terraform apply -> AWS resources -> ECS -> ALB -> endpoint -> LIVE.
Retry live test	AFTER GREEN DEPLOY	Confirm a new attempt safely reuses/reconciles foundation and does not inherit stale authority.
Rollback / Reset Fresh	SEPARATE CONTROLLED TESTS	Prove failure recovery and destructive recovery independently after normal deploy path is green.

7. Immediate Team Action Plan

1. Publish the corrected reusable workflow to the DeployGuard repository as a new immutable commit.
2. Update DeployGuard configuration to pin the exact new workflow SHA and verify local/remote hashes and input-contract parity.
3. Restart/reload DeployGuard under Node 22 with the new pin.
4. Run one normal fresh deployment using a safe test project. Do not use Reset Fresh for the first proof.
5. Capture evidence for Terraform plan/apply, resource creation/reconciliation, ECS service/task health, ALB target health, public endpoint response and stable-release ingestion.
6. If the normal deployment is green, test Retry as a separate case. Then test rollback and Reset Fresh separately so failures are attributable to one path.

> **Do not do**
> Do not delete Terraform state, EFS, database data, backups or historical attempts to make a deployment pass. Do not redesign the deployment architecture again unless live evidence exposes a genuine design contradiction. The current priority is publishing the corrected workflow and proving the real path.

8. Team Takeaway

What changed most is not the number of features; it is the consistency of the system's sources of truth. Environment identity, BuildPlan, deployment mode and persistence state are now intended to be decided once and carried across the pipeline rather than guessed independently by each stage.

- The old DB/EFS failure was a stale-state inference bug, not a Terraform initialization problem.
- The latest fix prevents stale secrets/bindings from impersonating a successful persistent database generation.
- Real persistent data remains protected.
- Repeated deployments are regression-tested to avoid old-attempt contamination.
- The remaining gap is live certification of the corrected immutable workflow against GitHub Actions and AWS.

Appendix A - Key Evidence from the Latest Implementation Log

Repository-analysis repair: Four iterations completed: canonical environment, authoritative BuildPlan, mainstream detector/template alignment, and complete repository-analysis certification.

Framework coverage: 24 mainstream framework/mode fixtures were certified through persisted contract creation, Docker generation, workflow serialization and readiness evaluation.

Cross-stage handoff: Repository fixture -> detection -> persisted BuildPlan -> Docker contract -> serialized workflow input -> runtime snapshot -> Terraform -> ECS port/health configuration.

Security boundary: Platform/repository-owned variables are filtered; secret values are not propagated into manifests/evidence; unsafe Dockerfile ARG/ENV secret use remains blocked.

Local startup: Control plane aligned to Node 22; product:start/product:verify and readiness checks passed; Redis/BullMQ remain intentionally inactive because GitHub Actions is the active deployment engine.

Live limitation: No live AWS resources or customer repositories were modified during these repair/certification steps.

Report basis: DeployGuard implementation logs and deployment failure/resolution evidence supplied by the project team on 10 August 2026. This report intentionally distinguishes local/test verification from live AWS proof.